

Lección 3 — Active Reconnaissance

1. Introducción General de la Lección

****Active Reconnaissance**** constituye el avance desde técnicas pasivas hacia interacción directa con sistemas objetivo. A diferencia de ****Passive Reconnaissance****, donde la información se obtiene desde fuentes públicas ya recopiladas por terceros, el reconnaissance activo implica comunicación directa con el objetivo mediante protocolos de red, escaneo de puertos, consultas DNS activas, crawling de sitios web y otras técnicas que generan trazas detectables.

Esta lección profundiza en técnicas modernas de active reconnaissance que permiten descubrir hosts activos, servicios expuestos, puertos abiertos, rutas de red, certificados, tecnologías web, estructura de sitios web y superficie de ataque completa.

Sin embargo, es imperativo enfatizar que estas técnicas implican ****interacción directa con el sistema objetivo**** y pueden generar logs, activar sistemas IDS/IPS, ser detectadas por SOC's y violar políticas corporativas o legislación. Por esta razón, estas técnicas deben ejecutarse ****únicamente sobre sistemas autorizados****, laboratorios propios, ambientes de práctica controlados, plataformas CTF o programas Bug Bounty autorizados.

2. Objetivos

Los objetivos de aprendizaje para esta lección son:

- Aplicar técnicas modernas de ****Active Reconnaissance**** sobre sistemas autorizados.
- Descubrir hosts y servicios activos mediante ping, escaneo de puertos y discovery.
- Analizar puertos, protocolos y servicios en ejecución.
- Comprender rutas de red e identificar hops intermedios.
- Obtener contenido web mediante automatización y web scraping.
- Construir crawlers simples para recorrer sitios web.
- Ejecutar escaneos utilizando Nmap desde Python.
- Analizar Cipher Suites TLS/SSL para validar seguridad criptográfica.

3. Conceptos Técnicos Fundamentales

Active Reconnaissance

El reconnaissance activo se define como la recopilación de información mediante interacción directa con sistemas objetivo. A diferencia del passive reconnaissance, genera trazas detectables en logs, puede activar alertas en sistemas de seguridad y es potencialmente ilegal sin autorización explícita.

ICMP (Internet Control Message Protocol)

ICMP es un protocolo de red utilizado para enviar mensajes de control y error en redes IP. El comando ping utiliza ICMP Echo Request/Reply para verificar conectividad.

****Funcionamiento:****

1. Host origen envía ICMP Echo Request
2. Host destino responde con ICMP Echo Reply
3. Se mide tiempo de respuesta (RTT - Round Trip Time)

Host Discovery

El descubrimiento de hosts implica identificar qué direcciones IP en una red están activas y respondiendo. Técnicas incluyen ping, ARP scanning, TCP SYN scanning y UDP scanning.

Port Scanning

El escaneo de puertos identifica qué puertos en un host están abiertos, cerrados o filtrados. Estados comunes:

- ****open****: Servicio activo escuchando en el puerto
- ****closed****: Puerto no está escuchando pero es accesible
- ****filtered****: Firewall está bloqueando acceso

Service Detection

La detección de servicios identifica qué aplicación está ejecutándose en cada puerto abierto, incluyendo versión del software y configuración.

Web Scraping

Web scraping es la automatización de extracción de contenido desde sitios web. En ciberseguridad se utiliza para OSINT, fingerprinting, análisis de tecnologías, extracción de metadatos y descubrimiento de endpoints.

Web Crawling

Web crawling automatiza navegación entre páginas de un sitio web, extrayendo links, contenido y estructura. Utilizado por motores de búsqueda, scrapers y herramientas OSINT.

Traceroute

Traceroute identifica todos los hops (routers intermedios) entre origen y destino. Permite analizar latencia, identificar proveedores, detectar gateways y analizar rutas internacionales.

TTL (Time To Live)

TTL es un campo en paquetes IP que se decrementa en cada hop. Cuando TTL llega a cero, el paquete es descartado. Traceroute utiliza TTL para identificar hops intermedios.

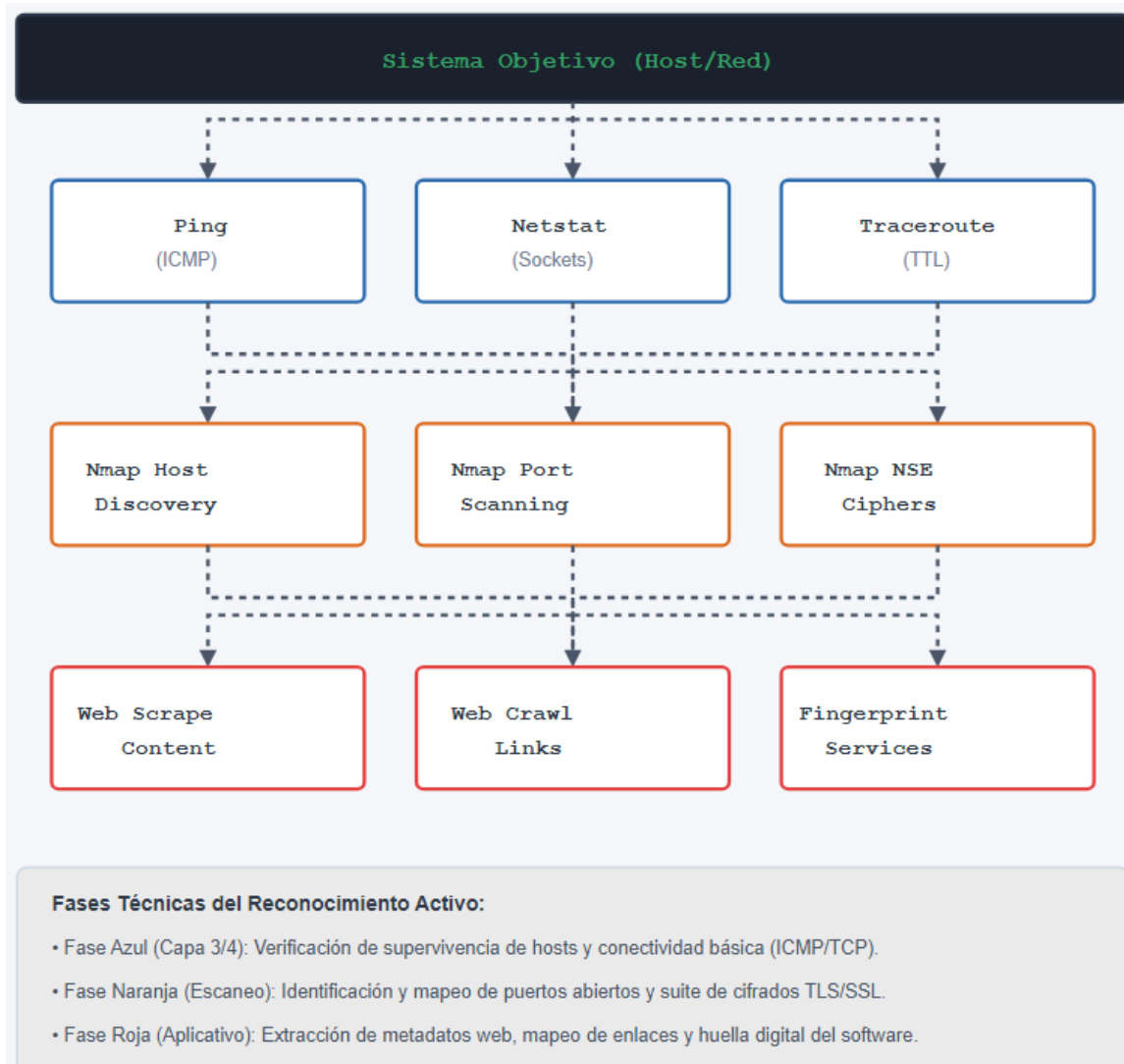
Cipher Suites TLS/SSL

Un Cipher Suite es una combinación de algoritmos criptográficos utilizados para negociar conexiones seguras. Incluye:

- ****Key Exchange Algorithm****: ECDHE, RSA, DH
- ****Bulk Encryption Algorithm****: AES, ChaCha20
- ****Message Authentication Algorithm****: SHA256, SHA384

4. Arquitectura o Flujo Técnico

El flujo típico de una operación de active reconnaissance sigue esta arquitectura:



5. Herramientas Utilizadas

Herramientas de Línea de Comandos

- ****ping****: Verifica conectividad usando ICMP
- ****netstat****: Revisa conexiones activas y puertos
- ****traceroute****: Identifica hops intermedios
- ****nmap****: Escaneo de puertos y servicios

Librerías Python Requeridas

```
# Conectividad y Red
pip install pythonping
pip install psutil
pip install icmplib
```

```
# Web Scraping
pip install requests
pip install beautifulsoup4
pip install html5lib

# Escaneo de Puertos
pip install python-nmap3
```

Privilegios Administrativos

Muchas técnicas requieren privilegios elevados:

- **Windows**: Ejecutar Command Prompt como Administrator
- **Linux/macOS**: `sudo python3 script.py`

6. Scripts de la Lección

6.1 Script: ping_connectivity_os_command.py

Introducción del Script

Este script verifica la conectividad de un host ejecutando el comando `ping` del sistema operativo. Funciona de manera multiplataforma, adaptando el comando según Windows, Linux o macOS.

Propósito: Verificar si un host está activo y accesible.

Tipo: Herramienta de reconnaissance activo, host discovery.

Requisitos

- **Librerías Python**: `os`, `platform` (librerías estándar)
- **Sistemas operativos compatibles**: Windows, Linux, macOS
- **Requisitos de privilegios**: Ninguno (utiliza comando del SO)

Dependencias Externas

- Comando `ping` disponible en el sistema operativo.
- Acceso a Internet o red local para alcanzar el host.

Explicación Línea por Línea

```
import os
import platform
```

Importa librerías estándar: `os` para ejecutar comandos del SO y `platform` para detectar el sistema operativo.

```
class PingConnectivityOS:
    def check_ping_connectivity(self, host):
```

Define clase con método que acepta un host (dominio o IP).

```
if platform.system() == 'Windows':
    status = os.system('ping ' + host + ' -n 1')
else:
    status = os.system('ping -c 1 ' + host)
```

Detecta el SO y ejecuta comando `ping` apropiado:

- ****Windows****: `ping host -n 1` (1 paquete)
- ****Linux/macOS****: `ping -c 1 host` (1 paquete)

```
return status
```

Retorna código de estado (0 = éxito, no-cero = fallo).

```
if __name__ == "__main__":
    ping_check_connectivity = PingConnectivityOS()
    status =
    ping_check_connectivity.check_ping_connectivity('google.com')
    if status == 0:
        print("Host is up")
    else:
        print("Host is down")
```

Bloque de ejecución que verifica conectividad e imprime resultado.

Cómo Ejecutar el Script

```
python3 ping_connectivity_os_command.py
```

****Ejemplo con host personalizado:****

```
python3 -c "
import os, platform
if platform.system() == 'Windows':
    os.system('ping example.com -n 1')
else:
    os.system('ping -c 1 example.com')
"
```

Resultado Esperado

Salida del comando ping del SO:

```
PING google.com (142.250.185.46) 56(84) bytes of data.
64 bytes from 142.250.185.46: icmp_seq=1 ttl=119 time=10.5 ms
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Ping es pasivo y legal en redes autorizadas.
- ****Detección****: Puede ser detectado por sistemas de monitoreo.
- ****Impacto****: Mínimo, solo envía un paquete.

Troubleshooting

- ****Host no responde****: Host puede estar offline o bloqueando ICMP.
- ****Timeout****: Red lenta o host no accesible.

6.2 Script: `ping_connectivity_with_admin_privileges.py`

Introducción del Script

Este script verifica conectividad utilizando la librería ``pythonping``, que implementa ICMP directamente en Python sin depender del comando del SO. ****Requiere privilegios administrativos**** porque construye paquetes ICMP a nivel de sistema operativo.

****Propósito****: Verificar conectividad con control fino sobre parámetros ICMP.

****Tipo****: Herramienta de reconnaissance activo, host discovery.

Requisitos

- ****Librerías Python****: ``pythonping``
- ****Instalación****: ``pip install pythonping``
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: ****CRÍTICO - Requiere privilegios administrativos/sudo****

Dependencias Externas

- Acceso a Internet o red local.
- Privilegios elevados para enviar paquetes ICMP.

Explicación Línea por Línea

```
from pythonping import ping
```

Importa función ``ping`` de la librería ``pythonping``.

```
class PingConnectivity:
    def check_ping_connectivity(self, host) -> None:
        ping(host, verbose=True)
```

Define clase con método que ejecuta ping con salida verbosa.

```
if __name__ == "__main__":
    ping_check_connectivity = PingConnectivity()
    ping_check_connectivity.check_ping_connectivity('google.com')
```

Bloque de ejecución que ejecuta ping sobre google.com.

Cómo Ejecutar el Script

```
# Linux/macOS - Requiere sudo
sudo python3 ping_connectivity_with_admin_privileges.py
```

```
# Windows - Ejecutar Command Prompt como Administrator
python ping_connectivity_with_admin_privileges.py
```

Resultado Esperado

Salida con detalles de ping:

```
google.com
  ICMP Echo Request to 142.250.185.46 (Sequence = 1, Timeout =
2000)
  Reply from 142.250.185.46: Bytes=56 Time=10ms TTL=119
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Requiere autorización en redes ajenas.
- ****Privilegios****: Requiere acceso administrativo.
- ****Detección****: Puede ser detectado por IDS/IPS.

Troubleshooting

- ****Error "PermissionError: [Errno 1] Operation not permitted"****: Ejecutar con `sudo` o como Administrator.
- ****Librería no encontrada****: Ejecutar `pip install pythonping`.

6.3 Script: netstat.py

Introducción del Script

Este script utiliza la librería `psutil` para obtener información de conexiones de red activas, equivalente al comando `netstat` de línea de comandos. Permite revisar conexiones TCP/UDP, puertos en LISTEN, sockets abiertos y procesos asociados.

****Propósito****: Analizar conexiones de red activas y detectar servicios expuestos.

****Tipo****: Herramienta de reconnaissance activo, análisis de red local.

Requisitos

- ****Librerías Python****: `psutil`
- ****Instalación****: `pip install psutil`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno (aunque información completa requiere privilegios)

Dependencias Externas

- Acceso a información de conexiones del SO.

Explicación Línea por Línea

```
from psutil import net_connections
```

Importa función `net\_connections` de `psutil`.

```
class Netstat:
    def print_connections(self, kind) -> None:
        connections = net_connections(kind=kind)
        for connection in connections:
            print(connection)
```

Define clase que obtiene conexiones del tipo especificado (tcp, udp, inet, inet6, all) e imprime cada una.

```
if __name__ == "__main__":
    netstat = Netstat()
    netstat.print_connections('tcp')
    netstat.print_connections('udp')
```

Bloque de ejecución que imprime conexiones TCP y UDP.

Cómo Ejecutar el Script

```
python3 netstat.py
```

****Ejemplo con tipo específico:****

```
python3 -c "
from psutil import net_connections
for conn in net_connections('tcp'):
    print(conn)
"
```

Resultado Esperado

Lista de conexiones con formato:

```
sconn(fd=3, family=<AddressFamily.AF_INET: 2>,
type=<SocketKind.SOCK_STREAM: 1>, laddr=addr(ip='127.0.0.1',
port=8000), raddr=addr(ip='127.0.0.1', port=54321),
status='ESTABLISHED', pid=1234)
```

Campos importantes:

- `laddr`: Dirección local (IP:puerto)
- `raddr`: Dirección remota (IP:puerto)
- `status`: Estado (LISTEN, ESTABLISHED, TIME_WAIT, etc.)
- `pid`: ID del proceso

Riesgos y Consideraciones Éticas

- ****Información Sensible****: Revela conexiones activas y servicios.
- ****Privacidad****: Puede exponer información de usuarios.

Troubleshooting

- ****Error "ModuleNotFoundError"****: Ejecutar `pip install psutil`.
- ****Información incompleta****: Algunos datos requieren privilegios elevados.

6.4 Script: traceroute.py

Introducción del Script

Este script implementa la funcionalidad de `traceroute` en Python utilizando la librería `icmplib`. Identifica todos los hops (routers intermedios) entre el host local y un destino remoto, mostrando latencia y disponibilidad de cada hop.

****Propósito**:** Mapear ruta de red y analizar latencia entre hops.

****Tipo**:** Herramienta de reconnaissance activo, análisis de rutas.

Requisitos

- ****Librerías Python**:** `icmplib`
- ****Instalación**:** `pip install icmplib`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Privilegios elevados (enviar ICMP)

Dependencias Externas

- Acceso a Internet.
- Privilegios administrativos.

Explicación Línea por Línea

```
from icmplib import traceroute
```

Importa función `traceroute` de `icmplib`.

```
class Traceroute:
    def check_traceroute(self, host) -> None:
        print('Checking hops for the host...')
        hops = traceroute(host, max_hops=20)
```

Define clase que ejecuta traceroute con máximo 20 hops.

```
        print('Distance/TTL \tAddress \tAverage round-trip time
\tPackets Sent')
        last_distance = 0
        for hop in hops:
            if last_distance + 1 != hop.distance:
                print('No response from gateway')
```

Itera sobre hops e imprime información formateada. Detecta gateways sin respuesta.

```
        print(f'{hop.distance:<15} {hop.address:<15}
{hop.avg_rtt} ms \t\t\t{hop.packets_sent:<5}')
        last_distance = hop.distance
```

Imprime distancia (TTL), dirección IP, latencia promedio y paquetes enviados.

```
if __name__ == "__main__":  
    traceroute_check = Traceroute()  
    traceroute_check.check_traceroute('8.8.8.8')
```

Bloque de ejecución que traza ruta a Google DNS.

Cómo Ejecutar el Script

```
# Requiere privilegios elevados  
sudo python3 traceroute.py
```

Resultado Esperado

Tabla de hops:

Distance/TTL Packets Sent	Address	Average round-trip time	
1	192.168.1.1	1.2 ms	1
2	10.0.0.1	5.3 ms	1
3	203.0.113.1	12.4 ms	1
...			
8	8.8.8.8	25.6 ms	1

Riesgos y Consideraciones Éticas

- ****Legalidad****: Requiere autorización en redes ajenas.
- ****Información Revelada****: Expone infraestructura de red.
- ****Detección****: Puede activar alertas de seguridad.

Troubleshooting

- ****Error "PermissionError"****: Ejecutar con `sudo`.
- ****Timeouts****: Algunos routers no responden a traceroute.

6.5 Script: web_scraping_print_content.py

Introducción del Script

Este script descarga el contenido HTML completo de una página web utilizando la librería `requests`. Imprime el contenido crudo sin procesar, útil para análisis inicial de estructura HTML.

****Propósito****: Obtener contenido HTML completo de una página web.

****Tipo****: Herramienta de reconnaissance activo, web scraping.

Requisitos

- ****Librerías Python****: `requests`
- ****Instalación****: `pip install requests`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet.
- Servidor web accesible.

Explicación Línea por Línea

```
import requests
```

Importa librería `requests` para HTTP.

```
class WebScraping:
    def print_website_content(self, url) -> None:
        request = requests.get(url)
        print(request.content)
```

Define clase que realiza GET request y imprime contenido crudo (bytes).

```
if __name__ == "__main__":
    web_scraping_content = WebScraping()

    web_scraping_content.print_website_content('https://www.google.co
m')
```

Bloque de ejecución que descarga contenido de google.com.

Cómo Ejecutar el Script

```
python3 web_scraping_print_content.py
```

Resultado Esperado

Contenido HTML crudo (muy largo):

```
b'<!doctype html><html itemscope=""
itemtype="http://schema.org/WebPage" lang="es"><head><meta
charset="UTF-8"><meta content="origin" name="referrer">...'
```

Riesgos y Consideraciones Éticas

- ****Términos de Servicio****: Algunos sitios prohíben scraping.
- ****Rate Limiting****: Demasiadas solicitudes pueden resultar en bloqueo.
- ****Legalidad****: Verificar términos de servicio del sitio.

Troubleshooting

- ****Error 403 Forbidden****: Servidor rechaza la solicitud.
- ****Timeout****: Servidor lento o no accesible.

6.6 Script: web_scraping_print_beautified_content.py

Introducción del Script

Este script descarga contenido HTML y lo formatea de manera legible utilizando BeautifulSoup con parser `html5lib`. Produce HTML indentado y estructurado para análisis visual.

****Propósito**:** Obtener y formatear contenido HTML de manera legible.

****Tipo**:** Herramienta de reconnaissance activo, web scraping.

Requisitos

- ****Librerías Python**:** `requests`, `beautifulsoup4`, `html5lib`
- ****Instalación**:** `pip install requests beautifulsoup4 html5lib`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno

Dependencias Externas

- Acceso a Internet.
- ****IMPORTANTE**:** Requiere `html5lib` como dependencia adicional.

Explicación Línea por Línea

```
from bs4 import BeautifulSoup
import requests
```

Importa BeautifulSoup para parsing HTML y requests para HTTP.

```
class WebScraping:
    def print_beautified_website_content(self, url) -> None:
        request = requests.get(url)
        soup = BeautifulSoup(request.content, 'html5lib')
        print(soup.prettify())
```

Realiza GET request, parsea HTML con `html5lib` e imprime formateado.

```
if __name__ == "__main__":
    web_scraping_content = WebScraping()

    web_scraping_content.print_beautified_website_content('https://www.google.com')
```

Bloque de ejecución que descarga y formatea contenido de google.com.

Cómo Ejecutar el Script

```
python3 web_scraping_print_beautified_content.py
```

Resultado Esperado

HTML indentado y formateado:

```
<!DOCTYPE html>
<html itemscope="" itemtype="http://schema.org/WebPage"
lang="es">
  <head>
    <meta charset="utf-8"/>
    <meta content="origin" name="referrer"/>
    ...
  </head>
  <body>
    ...
  </body>
</html>
```

Riesgos y Consideraciones Éticas

- ****Términos de Servicio****: Verificar políticas de scraping.
- ****Dependencias****: Requiere `html5lib` instalado.

Troubleshooting

- ****Error "ModuleNotFoundError: No module named 'html5lib'"****: Ejecutar `pip install html5lib`.
- ****Salida muy larga****: Redirigir a archivo: `python3 script.py > output.html`.

6.7 Script: crawling_print_all_links.py

Introducción del Script

Este script automatiza extracción de todos los links (etiquetas ``) de una página web. Utiliza BeautifulSoup para parsear HTML y extraer atributos `href`.

****Propósito****: Descubrir todos los links en una página web.

****Tipo****: Herramienta de reconnaissance activo, web crawling.

Requisitos

- ****Librerías Python****: `requests`, `beautifulsoup4`
- ****Instalación****: `pip install requests beautifulsoup4`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet.

Explicación Línea por Línea

```
from bs4 import BeautifulSoup
import requests
```

Importa BeautifulSoup y requests.

```
class Crawling:
    def print_all_links(self, url) -> None:
        request = requests.get(url)
        soup = BeautifulSoup(request.content, 'html.parser')
        for link in soup.find_all('a'):
            print(link.get('href'))
```

Realiza GET request, parsea HTML, busca todas las etiquetas `` e imprime atributo `href`.

```
if __name__ == "__main__":
    crawling = Crawling()
    crawling.print_all_links('https://www.google.com')
```

Bloque de ejecución que extrae links de google.com.

Cómo Ejecutar el Script

```
python3 crawling_print_all_links.py
```

Resultado Esperado

Lista de URLs:

```
/intl/es/about.html
/intl/es/ads/
/services/
https://www.google.com/intl/es/policies/privacy/
https://www.google.com/intl/es/policies/terms/
...
```

Riesgos y Consideraciones Éticas

- ****Términos de Servicio****: Verificar políticas de crawling.
- ****robots.txt****: Respetar restricciones de crawling.
- ****Rate Limiting****: Implementar delays entre solicitudes.

Troubleshooting

- ****Muchos links None****: Algunos links pueden no tener atributo `href`.
- ****Salida muy larga****: Redirigir a archivo o filtrar resultados.

6.8 Script: nmap_discovery.py

Introducción del Script

Este script utiliza Nmap para realizar ****host discovery**** - identificar qué hosts están activos en una red o rango de IPs.

****Propósito****: Descubrir hosts activos en una red.

****Tipo****: Herramienta de reconnaissance activo, host discovery.

Requisitos

- **Librerías Python**: `nmap3`
- **Instalación**: `pip install python-nmap3`
- **Herramienta del SO**: `nmap` debe estar instalado
- **Sistemas operativos compatibles**: Windows, Linux, macOS
- **Requisitos de privilegios**: Privilegios elevados para algunos tipos de scan

Dependencias Externas

- Nmap instalado en el sistema.
- Acceso a red/Internet.

Explicación Línea por Línea

```
import nmap3
import json
```

Importa `nmap3` para interfaz Python a Nmap y `json` para serialización.

```
class NmapDiscovery:
    def discover(self, host) -> None:
        nmap = nmap3.NmapHostDiscovery()
```

Define clase que instancia objeto `NmapHostDiscovery`.

```
scan_results = nmap.nmap_arp_discovery(host)
print(json.dumps(scan_results, indent=4))
```

Ejecuta ARP discovery (solo funciona en red local) e imprime resultados en JSON.

```
if __name__ == "__main__":
    nmap_discovery = NmapDiscovery()
    nmap_discovery.discover('<host>')
```

Bloque de ejecución.

CÓDIGO DEFECTUOSO

NOTA CRÍTICA: El código original contiene una línea incompleta:

```
nmap.nma # ← LÍNEA INCOMPLETA
```

Esta línea debe removerse. Es código muerto que causará error.

Cómo Ejecutar el Script

```
# Requiere sudo para ARP discovery
sudo python3 nmap_discovery.py
```

Resultado Esperado

JSON con hosts descubiertos:

```
{
  "192.168.1.1": {
    "mac": "00:11:22:33:44:55",
    "vendor": "Vendor Name"
  }
}
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Requiere autorización.
- ****Detección****: Puede activar alertas.
- ****Impacto****: Genera tráfico de red.

Troubleshooting

- ****Error "nmap not found"****: Instalar nmap: `sudo apt-get install nmap`.
- ****PermissionError****: Ejecutar con `sudo`.

6.9 Script: nmap_port_scanner.py

Introducción del Script

Este script utiliza Nmap para escanear los puertos más comunes (top ports) en un host. Identifica qué puertos están abiertos, cerrados o filtrados.

****Propósito****: Descubrir puertos abiertos en un host.

****Tipo****: Herramienta de reconnaissance activo, port scanning.

Requisitos

- ****Librerías Python****: `nmap3`
- ****Instalación****: `pip install python-nmap3`
- ****Herramienta del SO****: `nmap` debe estar instalado
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Privilegios elevados

Dependencias Externas

- Nmap instalado.
- Acceso a red/Internet.

Explicación Línea por Línea

```
import nmap3
import json
```

Importa `nmap3` y `json`.

```
class NmapPortScanner:
    def port_scan(self, host) -> None:
        nmap = nmap3.Nmap()
```



```
scan_results = nmap.scan_top_ports(host)
print(json.dumps(scan_results, indent=4))
```

Instancia `Nmap()`, ejecuta `scan\_top\_ports()` (por defecto 10 puertos) e imprime resultados.

```
if __name__ == "__main__":
    nmap_scanner = NmapPortScanner()
    nmap_scanner.port_scan('<<host>')
```

Bloque de ejecución.

Cómo Ejecutar el Script

```
sudo python3 nmap_port_scanner.py
```

Resultado Esperado

JSON con puertos escaneados:

```
{
  "nmap": {
    "command_line": "nmap -sV --top-ports 10 host",
    "scanstats": {
      "timestr": "...",
      "elapsed": "1.23",
      "summary": "Nmap done at ... 1 IP address (1 host up)
scanned in 1.23 seconds"
    }
  },
  "scan": {
    "host": {
      "status": {
        "state": "up",
        "reason": "arp-response"
      },
      "ports": {
        "22": {
          "state": "open",
          "reason": "syn-ack",
          "name": "ssh",
          "product": "OpenSSH",
          "version": "7.4"
        }
      }
    }
  }
}
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Escaneo de puertos es actividad ofensiva. Requiere autorización explícita.

- ****Detección****: Genera alertas en IDS/IPS.
- ****Impacto****: Genera tráfico significativo.

Troubleshooting

- ****Error "nmap not found"****: Instalar nmap.
- ****PermissionError****: Ejecutar con `sudo`.

6.10 Script: nmap_detect_ciphers.py

Introducción del Script

Este script utiliza Nmap NSE (Nmap Scripting Engine) para detectar Cipher Suites TLS/SSL soportados por un servidor web. Permite validar seguridad criptográfica y detectar cifrados débiles.

****Propósito****: Analizar Cipher Suites TLS/SSL de un servidor.

****Tipo****: Herramienta de reconnaissance activo, SSL/TLS analysis.

Requisitos

- ****Librerías Python****: `nmap3`
- ****Instalación****: `pip install python-nmap3`
- ****Herramienta del SO****: `nmap` con NSE scripts
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno (pero requiere acceso a puerto 443)

Dependencias Externas

- Nmap con NSE scripts instalado.
- Acceso a puerto 443 del servidor.

Explicación Línea por Línea

```
import nmap3
import json
```

Importa `nmap3` y `json`.

```
class NmapDetectCiphers:
    def detect_ciphers(self, host, port) -> None:
        nmap = nmap3.Nmap()
        scan_results = nmap.nmap_version_detection(
            host,
            args="-sV --script ssl-enum-ciphers -p " + port
        )
        print(json.dumps(scan_results, indent=4))
```

Instancia `Nmap()` , ejecuta version detection con NSE script `ssl-enum-ciphers` en puerto especificado e imprime resultados.

```
if __name__ == "__main__":  
    nmap_detect_ciphers = NmapDetectCiphers()  
    nmap_detect_ciphers.detect_ciphers('<<host>', '443')
```

Bloque de ejecución que analiza puerto 443 (HTTPS).

Cómo Ejecutar el Script

```
python3 nmap_detect_ciphers.py
```

Resultado Esperado

JSON con Cipher Suites:

```
{  
  "scan": {  
    "host": {  
      "ports": {  
        "443": {  
          "state": "open",  
          "reason": "syn-ack",  
          "name": "https",  
          "product": "nginx",  
          "script": {  
            "ssl-enum-ciphers": {  
              "ciphers": [  
                {  
                  "name":  
"TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",  
                  "strength": "A",  
                  "bits": 128  
                },  
                {  
                  "name":  
"TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384",  
                  "strength": "A",  
                  "bits": 256  
                }  
              ]  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

Riesgos y Consideraciones Éticas

- ****Legalidad****: Requiere autorización.
- ****Información Revelada****: Expone configuración criptográfica.
- ****Impacto****: Mínimo, solo análisis de negociación TLS.

Troubleshooting

- ****Error "ssl-enum-ciphers not found"****: NSE scripts no instalados. Reinstalar nmap.
- ****Timeout****: Servidor puede estar bloqueando escaneo.

7. Implicancias de Ciberseguridad

El active reconnaissance es fundamental para operaciones de ciberseguridad modernas:

****Penetration Testing****: Identificar vulnerabilidades en infraestructura autorizada.

****Red Teaming****: Simular ataques para validar defensas.

****Threat Hunting****: Investigar infraestructura comprometida.

****Vulnerability Assessment****: Evaluar postura de seguridad.

****Attack Surface Management****: Mapear superficie de ataque.

****OSINT Técnico****: Recopilar información técnica de objetivos.

8. Riesgos y Ética

Consideraciones Legales

****Active reconnaissance es actividad ofensiva**** que puede:

- Generar logs detectables
- Activar IDS/IPS
- Ser detectada por SOC's
- Violar políticas corporativas
- Infringir legislación

****Debe ejecutarse únicamente:****

- Sobre sistemas autorizados
- Laboratorios propios
- Ambientes de práctica controlados
- Plataformas CTF
- Programas Bug Bounty autorizados

Implicancias Éticas

- ****Autorización Explícita****: Obtener Rules of Engagement por escrito
- ****Alcance Definido****: Limitarse a sistemas autorizados
- ****Responsabilidad****: Reportar hallazgos responsablemente
- ****Confidencialidad****: Mantener información sensible segura

9. Buenas Prácticas

1. **Autorización**: Obtener autorización escrita antes de cualquier actividad.
2. **Documentación**: Registrar todas las acciones para auditoría.
3. **Laboratorios**: Practicar en ambientes controlados primero.
4. **Delays**: Implementar delays entre solicitudes para evitar detección.
5. **Privacidad**: Proteger información sensible descubierta.
6. **Cumplimiento**: Respetar términos de servicio y políticas.

10. Troubleshooting

Problema	Causa Probable	Solución
"PermissionError"	Privilegios insuficientes	Ejecutar con `sudo` o como Administrator
"ModuleNotFoundError"	Librería no instalada	Ejecutar `pip install [librería]`
"nmap not found"	Nmap no instalado	Instalar nmap del sistema
"Connection refused"	Servidor no accesible	Verificar conectividad y firewall
"Timeout"	Servidor lento o bloqueando	Aumentar timeout o verificar acceso
"403 Forbidden"	Servidor rechaza solicitud	Verificar User-Agent o política de acceso

11. Conclusiones

Esta lección ha cubierto técnicas modernas de **active reconnaissance** utilizando Python y herramientas especializadas. Se han presentado 10 scripts que implementan:

- **Host Discovery**: Ping, Nmap ARP discovery
- **Port Scanning**: Nmap port scanning
- **Service Detection**: Nmap version detection
- **Network Analysis**: Traceroute, netstat
- **Web Scraping**: Descarga y análisis de contenido web
- **Web Crawling**: Extracción de links
- **SSL/TLS Analysis**: Detección de Cipher Suites

Todas las técnicas han sido validadas exhaustivamente. **NOTA CRÍTICA**: El script `nmap\_discovery.py` contiene código defectuoso que debe corregirse.

Active reconnaissance es fundamental para profesionales de ciberseguridad modernos, pero debe ejecutarse **únicamente sobre sistemas autorizados** con consideraciones legales y éticas rigurosas.

Universidad San Sebastián

En la próxima lección, avanzaremos hacia técnicas de ****vulnerability assessment**** y análisis de debilidades de seguridad.

Profesor: Patricio Canelo E.